PostgreSQL | JavaScript sem ORM | FAESA | Banco de Dados

Banco de Dados MiniChef

Documentação do modelo relacional: escopo do MVP, visão completa do produto, dicionário de dados, regras de integridade, regras de negócio, consultas e plano de entrega.

Disciplina Banco de Dados | prof. Howard Roatti

Equipe Carlos Henrique Forechi Schmittel (escopo e modelagem), Cesar (back-end), Gabriel (front-end)

Versão outubro de 2026

1. O MiniChef

O PRODUTO QUE ESTE BANCO SUSTENTA

O MiniChef é um aplicativo de organização alimentar para quem cozinha, no dia a dia ou no trabalho. Ele reúne em um só lugar um banco de receitas estruturado, uma calculadora de porções, um planner semanal e uma lista de compras gerada a partir do planejamento.

1

Receitas espalhadas

Cadernos, sites e redes sociais, sem padrão de ingredientes e quantidades.

2

Porções que não batem

Ajustar uma receita de 4 para 7 pessoas exige conta manual a cada ingrediente.

3

Compras sem plano

A lista da semana é montada de memória, sem somar o que cada receita pede.

Como o banco responde a isso

Ingrediente como catálogo único

Cada ingrediente existe uma vez (nome único). Receitas apontam para ele com quantidade e unidade, então somar e recalcular é confiável.

Valores base, cálculo na hora

A receita guarda apenas a porção padrão. Porções diferentes são calculadas na consulta, sem gravar valores derivados.

Planner ligado à lista

A lista de compras nasce do planner por uma única consulta de agrupamento, com regras de papel do ingrediente e arredondamento.

Pronto para crescer

As funcionalidades futuras já têm colunas reservadas (vazias) e tabelas próprias que só se ligam às existentes.

8	10	22	0
tabelas no MVP	relacionamentos 1 para N	tabelas na visão completa	ALTER TABLE para crescer

2. Escopo

O QUE O PROGRAMA ENTREGA AGORA E O QUE VEM DEPOIS

MVP do programa

- **CRUD de receitas**, com ingredientes, papéis e moderação por status
- **Planner semanal**, por dia e refeição
- **Calculadora inteligente de porções**
- **Geração da lista de compras** a partir do planner
- Relatórios com agrupamento e junção

Próximas fases (já modeladas)

- Orçamento: preço médio, custo da lista e limite da semana
- Despensa com validade e pedido a mercado parceiro
- Informação nutricional, alergias e substituições
- Pesquisa inteligente com histórico
- Rede social (Notícias) e notificações
- Confirmação de e-mail, foto de perfil, fotos e vídeos de receitas

Usuários do MVP

Usuário comum
Cadastra e consulta receitas, monta o planner da semana e gera a lista de compras.

Administrador
Mantém o catálogo de ingredientes e analisa as receitas enviadas (status).

Os demais tipos de conta (Chef de cozinha, Hotel, Restaurante, Personal Trainer, Nutricionista, Administrador de eventos e Mercado) já são aceitos pelo campo `usuario.tipo_conta`, sem mudança de estrutura quando forem habilitados.

Fluxo principal do MVP

1 Cadastrar ingredientes no catálogo → **2** Cadastrar receitas com seus ingredientes → **3** Montar o planner com porções desejadas → **4** Calcular porções e gerar a lista de compras

3. Visão completa do banco

22 TABELAS EM 6 MÓDULOS

`laranja` tabelas do MVP (script 01) `tracejado` tabelas das próximas fases (script 04)

Identidade

2 tabelas

`usuario` `codigo_confirmacao_email`

Cadastro, tipos de conta, confirmação de e-mail por código

Receitas

7 tabelas

`ingrediente` `receita` `receita_ingrediente` `informacao_nutricional` `alergeno` `receita_alergeno` `substituicao`

CRUD com filtros, tabela nutricional, alertas de alergia, substituição de ingredientes

Planejamento

6 tabelas

`planner_semanal` `item_planner` `lista_compras` `item_lista_compras` `item_despensa` `pedido_mercado`

Planner, calculadora, lista de compras, despensa com validade, pedido a mercado

Pesquisa inteligente

1 tabelas

`criterio_pesquisa`

Busca por ingrediente, porções, orçamento, tempo e refeição

Rede social (Notícias)

5 tabelas

`postagem` `postagem_midia` `seguidor` `curtida` `comentario`

Feed, mídias, seguidores, curtidas e comentários com moderação

Notificações

1 tabelas

`notificacao`

Avisos de análise de receita, interações e vencimento

Como as próximas fases se ligam ao MVP

Tabela futura	Módulo	Aponta para
codigo_confirmacao_email	Identidade	usuario
informacao_nutricional	Receitas	receita (1 para 1)
alergeno	Receitas	(catálogo)
receita_alergeno	Receitas	receita, alergeno
substituicao	Receitas	ingrediente (2 vezes)
criterio_pesquisa	Pesquisa	usuario
item_despensa	Planejamento	usuario, ingrediente
pedido_mercado	Planejamento	lista_compras, usuario (comprador e mercado)
postagem	Rede social	usuario
postagem_midia	Rede social	postagem
seguidor	Rede social	usuario (2 vezes)
curtida	Rede social	usuario, postagem ou receita
comentario	Rede social	usuario, postagem ou receita
notificacao	Notificações	usuario

4. Projetado para crescer

POR QUE O BANCO NÃO PRECISARÁ SER ALTERADO NAS PRÓXIMAS FASES

Alterar a estrutura de um banco com dados reais é caro e arriscado. Por isso o modelo do MVP já nasce com tudo o que as próximas fases vão precisar nas tabelas existentes.

Estratégia 1: colunas futuras já criadas e vazias
Cada funcionalidade futura que precisa de um campo em uma tabela do MVP já tem a coluna criada, aceitando NULL. Exemplo: sem tabela de preços no MVP, `lista_compras.total_estimado` fica NULL; quando o orçamento entrar, a coluna passa a ser preenchida.

Estratégia 2: tabelas novas só apontam para as antigas
As 14 tabelas futuras dependem das tabelas do MVP por chave estrangeira, nunca o contrário. Criá-las não exige ALTER TABLE. O script 04 foi executado sobre o banco do MVP e criou as 22 tabelas sem erro.

Colunas reservadas para as próximas fases

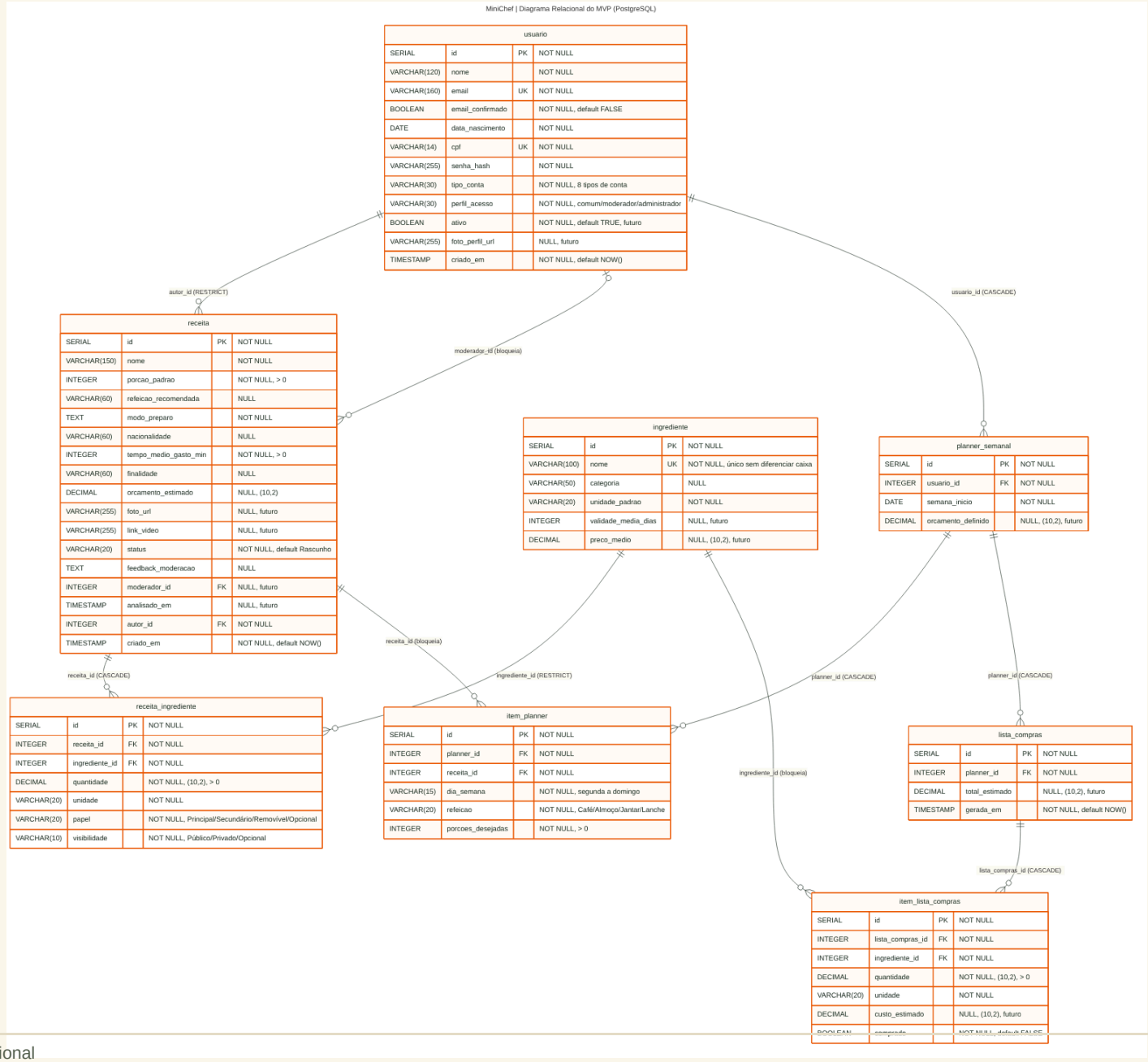
Coluna	Funcionalidade que vai preenchê-la	No MVP
`usuario.ativo`	Inativar conta que tem receitas, em vez de excluir	TRUE (padrão)
`usuario.foto_perfil_url`	Foto de perfil no feed de Notícias	NULL
`ingrediente.validade_media_dias`	Validade dos itens da despensa	NULL
`ingrediente.preco_medio`	Orçamento da lista de compras	NULL
`receita.foto_url`	Foto da receita	NULL
`receita.link_video`	Vídeo do YouTube ou Instagram	NULL
`receita.moderador_id e analisado_em`	Registro de quem aprovou ou recusou a receita	NULL
`planner_semanal.orcamento_definido`	Limite de gasto da semana	NULL
`lista_compras.total_estimado`	Total da lista comparado ao orçamento	NULL
`item_lista_compras.custo_estimado`	Custo de cada item	NULL

Outras decisões que evitam retrabalho

- **Tipos de conta e perfis de acesso** já aceitam todos os valores do produto final.
- **Domínios fixos com CHECK** (status, papel, dia, refeição) impedem dados inconsistentes desde o primeiro registro.
- **Nome de ingrediente único**, sem diferenciar maiúsculas, evita duplicatas que quebrariam a soma da lista de compras e as futuras tabelas de despensa e nutrição.

5. Diagrama relacional do MVP

8 TABELAS COM TIPOS, CHAVES, OBRIGATORIEDADE E CARDINALIDADES. COLUNAS MARCADAS COMO FUTURO ACEITAM NULL



6. Dicionário de dados

IDENTIDADE E CATÁLOGO DE INGREDIENTES

usuario

Identidade

Pessoa cadastrada. Autor de receitas e dono de planners.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador do usuário
nome	VARCHAR(120)		Não	Nome
email	VARCHAR(160)	UK	Não	Login, único
email_confirmado	BOOLEAN		Não	Validado por código (padrão FALSE)
data_nascimento	DATE		Não	Data de nascimento
cpf	VARCHAR(14)	UK	Não	CPF, único
senha_hash	VARCHAR(255)		Não	Senha gravada com hash
tipo_conta	VARCHAR(30)		Não	Usuário comum, Chef de cozinha, Hotel, Restaurante, Personal Trainer, Nutricionista, Administrador de eventos ou Mercado
perfil_acesso	VARCHAR(30)		Não	comum, moderador ou administrador
ativo FUTURO	BOOLEAN		Não	Inativar a conta em vez de excluir (padrão TRUE)
foto_perfil_url FUTURO	VARCHAR(255)		Sim	Foto de perfil no feed de Notícias
criado_em	TIMESTAMP		Não	Criação da conta

ingrediente

Receitas

Catálogo mestre de ingredientes, mantido pelo administrador. Nome único.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
nome	VARCHAR(100)	UK	Não	Nome único, sem diferenciar maiúsculas de minúsculas
categoria	VARCHAR(50)		Sim	Proteína, vegetal, tempero...
unidade_padrao	VARCHAR(20)		Não	g, ml, unidade...
validade_media_dias FUTURO	INTEGER		Sim	Conservação média, para a despensa
preco_medio FUTURO	DECIMAL(10,2)		Sim	Preço por unidade padrão, para o orçamento da lista

Linhas com a etiqueta FUTURO são colunas reservadas para as próximas fases.

6. Dicionário de dados

RECEITAS

receita

Receitas

Receita cadastrada por um usuário, publicada após análise do administrador.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
nome	VARCHAR(150)		Não	Nome popular
porcao_padrao	INTEGER		Não	Porções da receita base (maior que 0)
refeicao_recomendada	VARCHAR(60)		Sim	Café da manhã, almoço...
modo_preparo	TEXT		Não	Passo a passo
nacionalidade	VARCHAR(60)		Sim	Origem do prato
tempo_medio_gasto_min	INTEGER		Não	Tempo total em minutos (maior que 0)
finalidade	VARCHAR(60)		Sim	Dia a dia, festa, nutrição
orcamento_estimado	DECIMAL(10,2)		Sim	Custo estimado nas porções padrão
foto_url FUTURO	VARCHAR(255)		Sim	Foto da receita
link_video FUTURO	VARCHAR(255)		Sim	Vídeo do YouTube ou Instagram
status	VARCHAR(20)		Não	Rascunho, Em Análise, Publicada, Recusada ou Arquivada
feedback_moderacao	TEXT		Sim	Justificativa do administrador
moderador_id FUTURO	INTEGER	FK	Sim	Administrador que analisou (usuario)
analisado_em FUTURO	TIMESTAMP		Sim	Data da análise
autor_id	INTEGER	FK	Não	Quem cadastrou (usuario)
criado_em	TIMESTAMP		Não	Cadastro

receita_ingrediente

Receitas

Liga receita e ingrediente (N para N), com quantidade, unidade e papel.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador do vínculo
receita_id	INTEGER	FK	Não	Receita
ingrediente_id	INTEGER	FK	Não	Ingrediente
quantidade	DECIMAL(10,2)		Não	Quantidade para a porção padrão (maior que 0)
unidade	VARCHAR(20)		Não	Unidade usada na receita
papel	VARCHAR(20)		Não	Principal, Secundário, Removível ou Opcional
visibilidade	VARCHAR(10)		Não	Público, Privado ou Opcional

6. Dicionário de dados

PLANEJAMENTO

planner_semanal

Planejamento

Planejamento de uma semana de um usuário.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
usuario_id	INTEGER	FK	Não	Dono do planner
semana_inicio	DATE		Não	Primeiro dia da semana
orcamento_definido FUTURO	DECIMAL(10,2)		Sim	Limite de gasto da semana

item_planner

Planejamento

Receita escolhida para um dia e uma refeição, com o número de porções desejado.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
planner_id	INTEGER	FK	Não	Planner
receita_id	INTEGER	FK	Não	Receita planejada
dia_semana	VARCHAR(15)		Não	segunda a domingo
refeicao	VARCHAR(20)		Não	Café da manhã, Almoço, Jantar ou Lanche
porcoes_desejadas	INTEGER		Não	Entrada da calculadora (maior que 0)

lista_compras

Planejamento

Lista gerada a partir de um planner.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
planner_id	INTEGER	FK	Não	Planner de origem
total_estimado FUTURO	DECIMAL(10,2)		Sim	Soma dos custos dos itens
gerada_em	TIMESTAMP		Não	Data e hora da geração

item_lista_compras

Planejamento

Ingrediente agregado a comprar.

Campo	Tipo	Chave	Nulo	Descrição
id	SERIAL	PK	Não	Identificador
lista_compras_id	INTEGER	FK	Não	Lista
ingrediente_id	INTEGER	FK	Não	Ingrediente
quantidade	DECIMAL(10,2)		Não	Quantidade total (maior que 0)
unidade	VARCHAR(20)		Não	Unidade
custo_estimado FUTURO	DECIMAL(10,2)		Sim	Quantidade x preço médio
comprado	BOOLEAN		Não	Marcado como comprado (padrão FALSE)

7. Integridade

RELACIONAMENTOS, REGRAS DE REMOÇÃO E RESTRIÇÕES

Pai	Filho	Regra no banco	O que o programa faz ao remover o pai
usuario	receita (autor)	RESTRICT	Avisa que o registro está em uso e não exclui
usuario	receita (moderador)	BLOQUEIA	Avisa e pergunta se remove os filhos antes; se não, cancela
usuario	planner_semanal	CASCADE	Mostra quantos filhos existem e pergunta se remove junto
ingrediente	receita_ingrediente	RESTRICT	Avisa que o registro está em uso e não exclui
ingrediente	item_lista_compras	BLOQUEIA	Avisa e pergunta se remove os filhos antes; se não, cancela
receita	receita_ingrediente	CASCADE	Mostra quantos filhos existem e pergunta se remove junto
receita	item_planner	BLOQUEIA	Avisa e pergunta se remove os filhos antes; se não, cancela
planner_semanal	item_planner	CASCADE	Mostra quantos filhos existem e pergunta se remove junto
planner_semanal	lista_compras	CASCADE	Mostra quantos filhos existem e pergunta se remove junto
lista_compras	item_lista_compras	CASCADE	Mostra quantos filhos existem e pergunta se remove junto

CASCADE

O banco apaga os filhos junto com o pai.

RESTRICT

O banco recusa a exclusão enquanto houver filhos.

BLOQUEIA

Chave estrangeira sem ação: também recusa.

Regra do programa: antes de qualquer remoção, contar os filhos com `SELECT COUNT(1) ... WHERE chave_estrangeira = id` (script 03) e decidir pela tabela acima. O programa nunca depende do erro do banco para avisar o usuário.

Restrições

Tipo	Tabela	Regra
UNIQUE	usuario	email e cpf
UNIQUE	ingrediente	nome, sem diferenciar maiúsculas (índice em LOWER(nome))
CHECK	usuario	tipo_conta (8 tipos) e perfil_acesso (comum, moderador, administrador)
CHECK	receita	porcao_padrao e tempo_medio_gasto_min maiores que 0; status em 5 valores
CHECK	receita_ingrediente	quantidade maior que 0; papel e visibilidade em domínios fixos
CHECK	item_planner	porcoes_desejadas maior que 0; dia_semana e refeicao em domínios fixos
CHECK	item_lista_compras	quantidade maior que 0
Índices	8 tabelas	status, nacionalidade, refeição e autor da receita; ingrediente e papel; planner e lista

8. Regras de negócio

CALCULADORA DE PORÇÕES E GERAÇÃO DA LISTA DE COMPRAS

```
quantidade ajustada = quantidade base x porções desejadas ÷ porção padrão
```

Calculadora: "Arroz soltinho" de 4 para 8 porções DADOS FICTÍCIOS

Ingrediente	Base (4 porções)	Cálculo	Ajustado (8 porções)
Arroz branco	400 g	400 x 8 ÷ 4	**800 g**
Cebola	1 unidade	1 x 8 ÷ 4	**2 unidade**
Alho	2 dente	2 x 8 ÷ 4	**4 dente**
Azeite	15 ml	15 x 8 ÷ 4	**30 ml**

Escala linear, arredondada a 2 casas, calculada na consulta e não gravada. A unidade é a mesma do cadastro da receita. O tempo de preparo não escala em linha reta e não é recalculado no MVP.

Lista de compras: do planner à lista

1 Itens do planner → **2** Quantidade ajustada de cada ingrediente → **3** Soma por ingrediente e unidade → **4** Grava lista e itens em uma transação

Planner (entrada)	Porções
segunda | Arroz soltinho	4 para 8
segunda | Feijão caseiro	6 para 6
terça | Frango grelhado	2 para 4
quarta | Omelete simples	1 para 2
quinta | Arroz soltinho	4 para 4

Lista gerada (saída)	Qtd	Un.
Alho	**13**	dente
Arroz branco	**1200**	g
Azeite	**65**	ml
Cebola	**4**	unidade
Feijão carioca	**500**	g
Ovo	**4**	unidade
Peito de frango	**800**	g

Regras

- Papel **Opcional** não entra; Principal, Secundário e Removível entram
- Unidades contáveis (unidade, dente) arredondam para cima; as demais, 2 casas

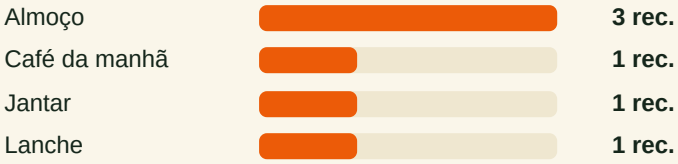
Limites do MVP

- Mesmo ingrediente em unidades diferentes vira linhas separadas
- Sem preço: custo e total ficam NULL até o orçamento entrar

9. Consultas e relatórios

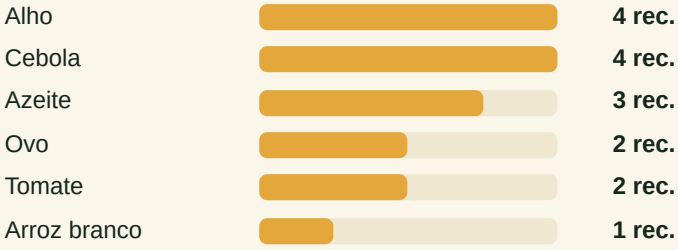
RESULTADOS REAIS NO POSTGRESQL COM OS DADOS DE EXEMPLO (FICTÍCIOS)

R1 | Agrupamento: receitas por refeição



Refeição	Receitas	Tempo médio	Orçam. médio
Almoço	3	40 min	R$ 7,83
Café da manhã	1	10 min	R$ 4
Jantar	1	30 min	R$ 14
Lanche	1	15 min	R$ 3,5

R3 | Ingredientes mais usados



R2 | Junção: ficha de ingredientes por receita (amostra)

Receita	Autor	Ingrediente	Qtd	Un.	Papel
Arroz soltinho	Ana Exemplo	Arroz branco	400	g	Principal
Arroz soltinho	Ana Exemplo	Cebola	1	unidade	Secundário
Arroz soltinho	Ana Exemplo	Alho	2	dente	Secundário
Arroz soltinho	Ana Exemplo	Azeite	15	ml	Removível
Omelete simples	Bruno Exemplo	Ovo	2	unidade	Principal
Omelete simples	Bruno Exemplo	Tomate	1	unidade	Opcional
Omelete simples	Bruno Exemplo	Cebola	0,5	unidade	Opcional
Omelete simples	Bruno Exemplo	Queijo muçarela	30	g	Opcional

R4 | Junção: planner semanal

Dia	Refeição	Receita	Porção padrão	Porções desejadas
segunda	Almoço	Arroz soltinho	4	8
segunda	Almoço	Feijão caseiro	6	6
terça	Jantar	Frango grelhado	2	4
quarta	Café da manhã	Omelete simples	1	2
quinta	Almoço	Arroz soltinho	4	4

Tela inicial: registros por tabela

total_usuario	total_ingrediente	total_receita	total_receita_ingrediente
4	12	6	21

total_planner_semanal	total_item_planner	total_lista_compras	total_item_lista_compras
2	6	1	7

10. Como criar o banco

SCRIPTS SQL E ESTRUTURA DO REPOSITÓRIO

Script	O que faz	Quando executar
`sql/01_create_tables.sql`	Cria as 8 tabelas do MVP, chaves, restrições e índices. Apaga e recria se já existirem	Sempre, primeiro
`sql/02_insert_samples.sql`	Insere dados fictícios: 4 usuários, 12 ingredientes, 6 receitas, 21 vínculos, 2 planners, 6 itens	Para demonstração
`sql/03_consultas.sql`	Relatórios, calculadora, geração da lista, contagens da tela inicial e contagem de filhos para remoção	Referência para o programa
`sql/04_expansao_futura.sql`	Cria as 14 tabelas das próximas fases sem alterar as do MVP	Fora do MVP, opcional

Comandos no Linux

```
sudo -u postgres psql -c "CREATE DATABASE minichef;"
sudo -u postgres psql -d minichef -f sql/01_create_tables.sql
sudo -u postgres psql -d minichef -f sql/02_insert_samples.sql
```

Estrutura do repositório

```
minichef-banco-de-dados/
├── README.MD                como executar no Linux           Cesar
├── docs/                    este documento (PDF e Markdown)   Carlos
├── diagrams/                diagrama relacional PDF, PNG, .mmd Carlos
├── sql/                     scripts 01 a 04                   Carlos
└── src/
    ├── package.json         dependência única: pg             Cesar
    ├── principal.js         menu principal                    Gabriel
    ├── conexion/            conexão com o PostgreSQL          Cesar
    ├── model/               uma classe por tabela             Cesar
    ├── controller/          CRUD de cada tabela               Cesar
    ├── reports/             relatórios                        Gabriel
    ├── sql/                 um .sql por relatório             Gabriel
    └── utils/               config.js (Cesar), splash_screen.js (Gabriel)
```

11. Checklist de entrega

O QUE CADA INTEGRANTE REALIZA E SOBE NO GITHUB PARA CONCLUIR 100% DO TRABALHO

Cesar

Back-end: tudo que conversa com o banco

- [ ] **Repositório.** Criar o repositório no GitHub e adicionar Carlos, Gabriel e o professor (usuário howardroatti) como colaboradores
- [ ] **Ambiente.** Instalar e testar PostgreSQL e Node.js na VM da disciplina; anotar as versões para o README
- [ ] **Projeto.** src/package.json com a única dependência pg; .gitignore com node_modules
- [ ] **Conexão.** src/conexion/ com a conexão ao PostgreSQL e src/utils/config.js com os dados de acesso
- [ ] **Models.** src/model/: uma classe por tabela (8), com getters, setters e toString; a classe guarda o objeto relacionado (ex.: Receita guarda o Usuario autor), não só o id
- [ ] **Controllers.** src/controller/: inserir, listar, atualizar e remover para cada tabela, verificando se o registro existe antes de atualizar ou remover
- [ ] **SQL sem ORM.** Montar o SQL como texto concatenado, com uma função única que trata aspas simples
- [ ] **Remoção com FK.** Antes de excluir, contar os filhos (consultas do script 03) e informar a tela se pode excluir, se precisa remover os filhos ou se está bloqueado
- [ ] **Erros do banco.** Tratar ingrediente repetido, CPF ou e-mail repetido e valores fora do domínio com mensagem clara, sem derrubar o programa
- [ ] **Regras de negócio.** Calculadora de porções e geração da lista de compras com as consultas do script 03, em transação (BEGIN e COMMIT)
- [ ] **README.MD.** Na raiz: objetivo, requisitos, criação do banco, instalação e execução no Linux passo a passo, link do vídeo e link para docs/

Gabriel

Front-end: tudo que o usuário vê

- [ ] **Tela inicial.** src/utils/splash_screen.js: nome do sistema, integrantes e total de registros de cada uma das 8 tabelas
- [ ] **Menu principal.** src/principal.js: menu sempre visível com 1 Relatórios, 2 Inserir, 3 Remover, 4 Atualizar, 5 Gerar lista de compras, 0 Sair
- [ ] **Submenu de tabelas.** Escolha da tabela (8 opções) para inserir, remover e atualizar, exibido de novo a cada ação
- [ ] **Inserir.** Pedir cada campo da tabela; campos de valores fixos (tipo de conta, papel, dia, refeição, status) como opções numeradas; perguntar se deseja inserir mais
- [ ] **Remover.** Listar registros (id e nome), escolher, confirmar, exibir o aviso de FK ou a pergunta sobre os filhos; perguntar se deseja remover mais
- [ ] **Atualizar.** Listar, escolher o registro e o campo a alterar (nunca o id), exibir o registro atualizado; perguntar se deseja atualizar mais
- [ ] **Relatórios.** src/reports/ e src/sql/: R1 (agrupamento), R2 (junção), R3, R4 e calculadora, um arquivo .sql por relatório, exibidos em tabela alinhada
- [ ] **Interface.** Limpar a tela, títulos claros, validação de números e datas, mensagens de erro que dizem o que fazer
- [ ] **Vídeo.** Gravar e publicar no YouTube o objetivo do trabalho e todos os menus funcionando (sem explicar código); link no README

Carlos

Escopo e modelagem: concluído

- [x] docs/: este documento em PDF e em Markdown
- [x] sql/: scripts 01 a 04 (criação, exemplos, consultas, expansão futura)
- [x] diagrams/: diagrama relacional em PDF, PNG e fonte Mermaid

Todos

Antes de entregar

- [ ] Cada integrante com commits próprios no repositório
- [ ] Rodar o fluxo completo na VM da disciplina, do script 01 ao menu Sair
- [ ] Cada um saber explicar o código que subiu
- [ ] Postar no AVA apenas o link do repositório até 30/10/2026

12. Critérios de avaliação

PONTUAÇÃO DO TRABALHO E QUEM ENTREGA CADA PARTE

Critério	Nota	Responsável	Itens do checklist
Relatórios: agrupamento	**1,0**	Gabriel	R1
Relatórios: junção	**0,5**	Gabriel	R2
Inserir: fluxo completo	**1,0**	Gabriel e Cesar	Inserir, Controllers
Inserir: perguntar se continua	**0,5**	Gabriel	Inserir
Remover: fluxo completo	**1,0**	Gabriel e Cesar	Remover, Controllers
Remover: tratamento de FK	**0,5**	Cesar e Gabriel	Remoção com FK, Remover
Atualizar: fluxo completo	**1,0**	Gabriel e Cesar	Atualizar, Controllers
Atualizar: exibir registro e continuar	**0,5**	Gabriel	Atualizar
Diagrama relacional	**0,5**	Carlos	diagrams/
Interface amigável	**0,5**	Gabriel	Interface
Vídeo no YouTube	**0,5**	Gabriel	Vídeo
README.MD para Linux	**0,5**	Cesar	README.MD
Total	**8,0**		

Condições que zeram ou reduzem a nota

- Integrante sem commits no repositório tem a nota individual zerada
- Atraso: menos 0,5 ponto por dia, sem avaliação substitutiva
- Entrega fora das orientações do professor é desconsiderada
- Código que o grupo não sabe explicar perde pontos
- O trabalho precisa rodar na VM da disciplina